

Investigación Integral: Arquitectura de Software, POO y Sistemas de Datos

1. Introducción al Desarrollo de Sistemas

El desarrollo de software profesional no solo consiste en escribir código, sino en diseñar estructuras que puedan crecer y adaptarse. Este proceso se apoya en el paradigma de la **Programación Orientada a Objetos (POO)** y la persistencia de datos en sistemas relacionales.

2. Pilares de la Programación Orientada a Objetos (POO)

La POO nos permite modelar problemas complejos usando conceptos del mundo real.

1. **Abstracción:** Seleccionar solo las propiedades y comportamientos esenciales de un objeto para el contexto del sistema. Por ejemplo, en un sistema escolar, de una persona nos interesa su **ID** y **Notas**, no su **Color favorito**.
2. **Encapsulamiento:** Agrupar datos y métodos, restringiendo el acceso directo a los datos mediante modificadores (Private, Public, Protected). Esto se logra mediante métodos **Getters** (obtener) y **Setters** (modificar).
3. **Herencia:** Mecanismo donde una "Clase Hija" deriva de una "Clase Padre".
 - *Ejemplo:* **Persona** es la clase padre. **Alumno** y **Docente** son clases hijas que heredan el nombre y la edad, pero añaden sus propios datos.
4. **Polimorfismo:** La capacidad de que una misma operación se comporte distinto en diferentes clases. Un método **calcularDescuento()** funcionará diferente para un Alumno becado que para un Docente veterano.

3. Anatomía Profunda de una Clase

3.1. Atributos (El Estado)

Son las variables que definen a la clase. En el desarrollo profesional, se clasifican según su visibilidad:

- **Privados (-):** Solo accesibles dentro de la misma clase.
- **Públicos (+):** Accesibles desde cualquier parte del programa.
- **Protegidos (#):** Accesibles por la clase y sus descendientes (hijas).

3.2. Métodos (El Comportamiento)

Funciones que definen la lógica:

- **Constructores:** Inicializan el objeto al nacer (ej: `new Alumno("Juan")`).
- **Métodos de Negocio:** Realizan cálculos o procesos (ej: `inscribir()`).
- **Destruyores:** Liberan memoria cuando el objeto ya no se usa (común en lenguajes como C++).

4. Tipos de Clases según su Función Arquitectónica

En una arquitectura de capas (como MVC - Modelo Vista Controlador), las clases se dividen en:

- **Clases de Entidad (Modelos):** Representan los datos (Alumno, Materia).
- **Clases de Servicio (Lógica):** Contienen algoritmos complejos de procesamiento.
- **Clases de Repositorio (Acceso a Datos):** Se encargan exclusivamente de hablar con la base de datos (Guardar, Editar, Eliminar).
- **Clases de Utilidad:** Ofrecen herramientas genéricas (formatear fechas, validar correos).

5. Modelado del Sistema Académico Detallado

5.1. Clase Alumno

Es el eje central de la gestión educativa.

- **Atributos:**
 - `id_alumno` (Tipo: Entero Largo/PK)
 - `nombres_completos` (Tipo: Cadena de texto)
 - `correo_institucional` (Tipo: Cadena, debe ser único)
 - `promedio_ponderado` (Tipo: Flotante/Decimal)
- **Métodos:**
 - `inscribirEnCiclo(ciclo_id)`: Valida requisitos y registra.
 - `validarCreditos()`: Revisa si el alumno tiene créditos suficientes para matricularse.

5.2. Clase Docente

Define al personal académico responsable de la enseñanza.

- **Atributos:**
 - `id_docente` (Tipo: Entero/PK)
 - `especialidad` (Tipo: Cadena)
 - `escalafon` (Tipo: Enumeración: Junior, Senior, Titular)
 - `sueldo_base` (Tipo: Moneda)
- **Métodos:**

- `registrarCalificaciones(materia_id, listado_notas)`: Impacta la base de datos.
- `asignarHorario(bloque_id)`: Verifica que no haya cruces con otras clases.

5.3. Clase Materia

Es el componente curricular que vincula a alumnos y docentes.

- **Atributos:**
 - `codigo_materia` (Tipo: Cadena/PK)
 - `nombre` (Tipo: Cadena)
 - `total_horas_semestre` (Tipo: Entero)
 - `cupos_maximo` (Tipo: Entero)
- **Métodos:**
 - `estaLlena()`: Devuelve verdadero si ya no hay cupos.
 - `obtenerPreRequisitos()`: Consulta qué materias deben estar aprobadas antes.

6. Integración con Bases de Datos Relacionales

Para que los datos de las clases no se pierdan al cerrar el programa, usamos una base de datos.

6.1. El Proceso de Mapeo (ORM)

El **Object-Relational Mapping** es la técnica de convertir los objetos de código en registros de tablas.

- **Clase -> Tabla**
- **Atributo -> Columna**
- **Objeto -> Fila/Registro**

6.2. Diseño de Tablas y Relaciones

Para que el sistema sea eficiente, las tablas deben estar relacionadas:

1. **Relación Alumno - Materia (Muchos a Muchos)**: Un alumno lleva varias materias y una materia tiene muchos alumnos. Se crea una tabla intermedia llamada `MATRICULAS`.
2. **Relación Docente - Materia (Uno a Muchos)**: Un docente puede dictar varias materias, pero una materia (en un grupo específico) suele tener un solo docente responsable.

7. Ciclo de Vida del Desarrollo (SDLC)

Para implementar estas clases, se sigue un proceso ordenado:

1. **Análisis:** Definir qué atributos necesita cada clase.
2. **Diseño:** Crear diagramas de clases (UML) y diagramas de base de datos (DER).
3. **Implementación:** Escribir el código en lenguajes como C#, Java o Python.
4. **Pruebas:** Verificar que los métodos (como `calcularPromedio`) den resultados correctos.
5. **Mantenimiento:** Ajustar las clases si cambian las reglas del colegio o universidad.

8. Conclusión

El desarrollo de software y las bases de datos son dos caras de la misma moneda. Mientras que las **clases** dan vida a la lógica y permiten que el programa funcione, las **bases de datos** aseguran que la información sea segura, organizada y persistente en el tiempo. Un buen programador debe dominar ambos mundos para construir sistemas robustos y confiables.